Segmentation

Week 8 — Session 1: Thresholds & regions

Practical Image Processing

Lecture 8.1 ▪ 60–70 min

Recap & today

- Wk2/7: thresholding + morphology give clean binary shapes.
- Today (Session 1): **segmentation** — partitioning an image into meaningful regions.
- We start where you already are (thresholds) and grow into **region-based** methods.

DocScan milestone

Turn the meter photo into clean regions: page vs ink, then each digit as its own region — the base for reading them.

Roadmap

Why segmentation

Threshold-based segmentation

Region-based segmentation

In-class exercise

Wrap-up

Why segmentation

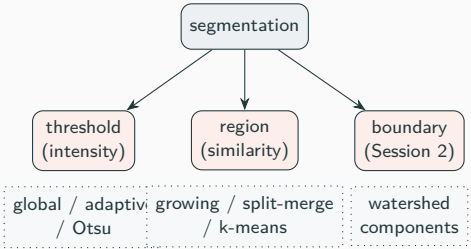
Segmentation = assign each pixel a label

- The goal: group pixels into regions that mean something (a digit, the page, the sky).
- Simplest case: two labels (object / background) = **thresholding** (Week 2).
- Harder: many objects, touching objects, or regions defined by colour/texture.

Key idea

Segmentation is where "pixels" become "objects." Everything downstream (counting, reading) depends on it.

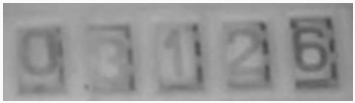
A map of the methods



- Session 1 covers the left two branches; Session 2 covers boundaries.

Threshold-based segmentation

Global vs adaptive threshold — on the meter



gray



global Otsu



adaptive

Otsu picks one global threshold (great for even lighting); adaptive computes a local one (robust to shadows). Recall Week 2 — this is our starting point.

Otsu, revisited

- Finds the threshold that best separates the two histogram modes (max between-class variance).
- Works when the histogram is clearly **bimodal** (object vs background).
- Fails on: uneven lighting, more than two classes, or a unimodal histogram.

Math

Maximize $\sigma_B^2(t) = w_0(t)\, w_1(t)\, [\mu_0(t) - \mu_1(t)]^2$ over the threshold t, where w_0, w_1 are the class probabilities and μ_0, μ_1 their means.

Otsu — a tiny worked example

A 4-level image (levels 0–3) with pixel counts:

level i	0	1	2	3
count	4	2	2	4
p_i	.33	.17	.17	.33

Try threshold $t = 1$ (class 0 = levels $\{0, 1\}$, class 1 = $\{2, 3\}$):

Math

$w_0 = .50,\ w_1 = .50,\ \mu_0 = \frac{0 \cdot .33 + 1 \cdot .17}{.50} = 0.33,\ \mu_1 = 2.5$

$\sigma_B^2(1) = .50 \cdot .50 \cdot (2.5 - 0.33)^2 = 0.25 \cdot 4.71 = 1.18$

Otsu — scanning every threshold

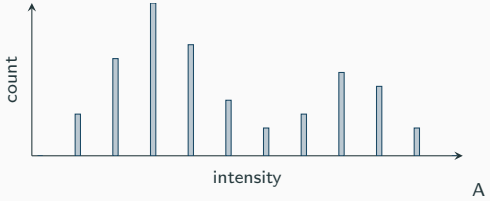
Compute $\sigma_B^2(t)$ for each candidate and keep the max:

t	w_0	$\mu_1 - \mu_0$	$\sigma_B^2(t)$
0	.33	1.75	0.68
1	.50	2.17	1.18
2	.67	1.75	0.68

✔ Good practice

The symmetric histogram peaks at $t = 1$ — exactly between the two modes. Otsu is just this scan, done over all 256 levels.

When a single threshold fails



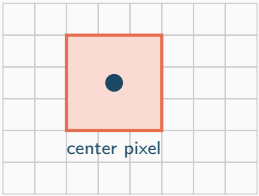
shadow spreads the object mode — no single cut separates it.

✗ Pitfall

Uneven lighting flattens the valley Otsu relies on. One global t then either eats the shadowed text or floods the bright page.

Fix: compute a threshold *locally*.

Adaptive thresholding — how it works



- Slide a window; threshold each pixel by the *local* mean (or Gaussian mean).
- $T(x, y) = \text{mean}_{\text{block}}(x, y) - C$.
- Big block $\rightarrow$ smoother; small $C \rightarrow$ more ink kept.

Math

Pixel is foreground if $I(x, y) < \text{mean}_{\text{block}}(x, y) - C$.

Adaptive threshold in code

```python
import cv2
gray = cv2.imread("meter.png", cv2.IMREAD_GRAYSCALE)

# Global Otsu (one threshold for the whole image)
_, otsu = cv2.threshold(gray, 0, 255,
                        cv2.THRESH_BINARY + cv2.THRESH_OTSU)

# Adaptive: local mean over an 31x31 block, minus C=10
adap = cv2.adaptiveThreshold(gray, 255,
        cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
        cv2.THRESH_BINARY, blockSize=31, C=10)
```

- `blockSize` must be odd; `C` shifts the decision brighter/darker.

Tuning adaptive threshold

✖ Pitfall

`blockSize` too small → noisy speckle; too large → behaves like a global threshold and shadows return.

✔ Good practice

Start block ≈ the stroke width × 3, then nudge `C` until the page is clean but strokes stay connected.

Key idea

Global Otsu answers "what is the best single cut?" Adaptive answers "what is the best cut *here*?" Lighting decides which you need.

Region-based segmentation

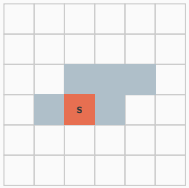
Beyond thresholds

- **Region growing**: start from seeds, add neighbours that are similar enough.
- **Split-and-merge**: divide the image, then merge regions that are alike.
- **Colour/k-means**: cluster pixels by colour into k regions.

Key idea

Thresholding uses only intensity; region methods use *similarity and connectivity* — more powerful, more parameters.

Region growing — the idea



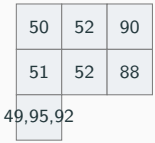
seed **s** grows into similar neighbours.

1. Choose seed pixel(s).
2. Add a 4/8-neighbour if $|I_{\text{nbr}} - \mu_{\text{region}}| \leq \tau$.
3. Update the region mean; repeat until nothing is added.

- Grows exactly one connected region per seed.

Region growing — worked step

Seed at the centre ($= 52$), tolerance $\tau = 8$. Add a neighbour if within ± 8 of the region mean:



- Start region $= \{52\}$, mean $= 52$. Neighbours $50, 52, 51, 95$: keep $50, 52, 51$ (≤ 8), reject 95.
- New mean ≈ 51.3; its neighbour 49 joins, $90/88/92/95$ stay out.
- Result: the left "≈ 50" region is captured; the right bright block is a separate region.

Split-and-merge (quadtree)



1. **Split**: if a block is not uniform (variance $> \tau$), divide into 4.
2. Recurse until every block is uniform.
3. **Merge**: adjacent blocks that are similar become one region.

- Adapts detail: big blocks in flat areas, tiny blocks at edges.

k-means colour segmentation — real car



original



k-means, k=4 colours

- Each pixel is a point in (R,G,B); k-means groups them into k clusters.

k-means: how it works

1. Pick k initial cluster centres.
2. Assign each pixel to the nearest centre.
3. Recompute each centre as the mean of its pixels.
4. Repeat 2–3 until stable.

✕ Pitfall

You must choose k; the wrong k over- or under-segments. It also ignores *position* — only colour.

k-means — a 1-D worked pass

Intensities $\{20, 24, 90, 96\}$, $k = 2$, initial centres $c_1 = 20$, $c_2 = 90$:

Math

Assign: $20, 24 \to c_1$; $90, 96 \to c_2$.

Update: $c_1 = \frac{20+24}{2} = 22$, $c_2 = \frac{90+96}{2} = 93$.

Re-assign: same membership $\Rightarrow$ converged in one pass.

- Final clusters: $\{20, 24\}$ (dark) and $\{90, 96\}$ (bright).
- In colour, the same runs in 3-D over (R,G,B) instead of 1-D.

k-means in code

```
Z = img.reshape((-1, 3)).astype("float32")   # N x 3 (RGB)
crit = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 20, 1.0)
k = 4
_, labels, centers = cv2.kmeans(Z, k, None, crit, 10,
                                cv2.KMEANS_RANDOM_CENTERS)
seg = centers[labels.flatten()].reshape(img.shape).astype("uint8")
```

- `labels` is the cluster id per pixel; `centers` the k mean colours.
- Run several inits (`attempts=10`) and keep the lowest-error one.

Choosing k

✗ Pitfall

Too small k: distinct objects merge. Too large k: one object splits into colour bands (shadow vs highlight).

✓ Good practice

Add spatial coordinates (x, y) to the feature vector so clusters stay spatially compact, or post-process with connected components (Session 2).

In-class exercise

In-class exercise (10 min)

Pick the threshold/region method

Which Session-1 method fits each task, and what parameter matters most?

1. Dark text on an evenly lit white page.
2. A document photographed with a shadow across half of it.
3. Segment a photo into “sky / road / car” by colour.
4. Grow the meter’s dial region outward from a click at its centre.

Debrief

- (1) Even lighting, bimodal $\rightarrow$ **Otsu**; parameter: none (auto).
- (2) Shadow $\rightarrow$ **adaptive**; parameter: `blockSize`, `C`.
- (3) Colour regions $\rightarrow$ **k-means** (or HSV colour thresholding); parameter: k.
- (4) Seeded region $\rightarrow$ **region growing**; parameter: tolerance τ.

Key idea

Match the method to the structure: intensity (threshold), lighting (adaptive), colour (k-means), or a known seed (region growing).

Wrap-up

What you can do now

- Threshold globally (Otsu) and locally (adaptive), and know when each fails.
- Hand-compute Otsu's between-class variance for a small histogram.
- Grow a region from a seed and split/merge by uniformity.
- Cluster colours with k-means and reason about the choice of k.

Next session

Session 2 — boundaries & objects

Threshold + region methods still merge *touching* objects into one blob. Next time: separate them with the **watershed**, label each with **connected components**, and measure quality with **IoU/Dice** — then the week's deliverable.

- Bring your Otsu-vs-adaptive meter results; we build on them.

Thresholds split by intensity, regions by similarity.

Otsu for even light, adaptive for shadows, k-means for colour, growing from seeds.